Dizital Adda LMS

Production-Level Architecture Audit

Generated from source inspection on 27/6/2026

1. Overall Project Summary

This is a React/Vite frontend with an Express/PostgreSQL backend. The project has visible LMS screens for landing pages, login, dashboards, courses, checkout, learning, teacher tools, and admin tools.

The actual backend start script runs backend/src/server.js. backend/src/app.js mounts additional routes but is not used by backend/package.json.

Many features are scaffolded or UI-only. Several routes/controllers exist but are not mounted in the runtime server. Database schema/migrations are not present in the repository.

2. Current Project Completion Percentage

Overall completion: approximately 28%.

The project is a beginner to early-intermediate prototype, not production ready.

Approximate remaining work: 72%. Estimated production-readiness work: 8-14 weeks for a small focused team, longer for a solo developer.

3. Frontend Completion Percentage

- Landing Website: 65% - many polished pages exist, but parts are static/demo content.
- Authentication: 35% - login UI and role redirects exist, but auth state is localStorage-only and not strongly verified.
- Student Portal: 30% - dashboard/learning UI exists, but real progress/enrollment/lectures are incomplete.
- Teacher Portal: 25% - dashboard, courses, assignments, upload lecture screens exist; most data is static or non-persistent.
- Admin Portal: 45% - users/courses/students/teachers/settings/payments pages exist; several calls are broken or unsecured.
- Responsive Design: 45% - Tailwind layouts exist, but no systematic responsive QA is visible.
- UI/UX Quality: 45% - visually ambitious, but inconsistent, alert-heavy, static, and not yet production polished.
- Overall Frontend: 38%. Build passes, lint fails with 64 errors.

4. Backend Completion Percentage

- Authentication: 25% - login exists, but JWT secrets are inconsistent and protected routes reject normal tokens.
- APIs: 30% - multiple route files exist, but many are incomplete, duplicated, unmounted, or unprotected.
- Controllers: 28% - CRUD scaffolds exist for several domains, but many workflows are incomplete.
- Middleware: 20% - basic token/role/upload middleware exists; security is not production grade.
- Validation: 10% - almost no schema validation.
- Database: 15% - code references tables, but schema/migrations are absent.
- Security: 10% - major auth/RBAC/secrets/upload/rate-limit gaps.
- Error Handling: 25% - basic try/catch responses exist, but no consistent error model.
- Overall Backend: 24%.

5. LMS Features Audit

- Authentication: 25% - login only; no registration/reset/refresh/email verification; token mismatch bug.
- Student Login: 20% - frontend role redirect exists; backend student password route stores plaintext.
- Teacher Login: 25% - teacher users can be added; no dedicated workflow.
- Admin Login: 30% - admin redirect exists; most admin APIs are unprotected.
- Course Management: 40% - add/list/detail/edit/delete partially exist; schemas and URLs are inconsistent.
- Categories: 5% - no real database/API category module.
- Chapters/Sections: 20% - section create controller exists but is not mounted in runtime server.
- Lectures: 15% - upload route returns success but does not persist or stream files.
- Video Player: 20% - UI exists; backend returns dummy/empty lecture data.
- Progress Tracking: 15% - controller references tables, but no schema/update flow.
- Continue Learning: 10% - no persisted resume state.
- Enrollments: 20% - controller exists but not mounted; contains undefined addActivity/course_id bug.
- Payments: 20% - Razorpay order creation exists; no verification, persistence, webhooks, or enrollment activation.
- Razorpay Integration: 20% - test/client integration only, not secure production flow.
- Orders: 5% - no order model/schema/workflow found.
- Certificates: 20% - generate/verify controller exists but is unmounted and schema absent.
- Assignments: 25% - backend scaffolds exist but unmounted; frontend mostly static.
- Quizzes: 25% - backend scaffolds exist but unmounted; no complete frontend workflow.
- Attendance: 0% - not implemented.
- Notes: 5% - PDF field exists, no notes feature.
- Downloads: 10% - report export UI exists; no secured material downloads.
- Reviews: 5% - static testimonials only.
- Notifications: 10% - static admin page only.
- Dashboard: 35% - some admin stats API exists; many dashboard numbers are static.
- Reports: 20% - static payment report generation UI; no real backend reporting.
- Analytics: 20% - count queries and static charts only.
- Settings: 20% - settings UI/password route exists; route is unprotected and frontend URL malformed.

6. Database Audit

No schema files, migrations, ORM models, seed files, or SQL DDL were found in the project source.

Tables referenced by code: users, students, courses, payments, enrollments, sections, quizzes, quiz_questions, quiz_attempts, assignments, assignment_submissions, certificates, activities, doubts, video_progress, test_results.

Missing tables cannot be proven from actual schema because schema is absent. Relationship quality cannot be verified. Code suggests weak normalization: students duplicates user-like fields, courses.teacher appears text-like, and users.name/users.full_name are inconsistent.

Database production readiness: 15%.

7. API Audit

- POST /api/auth/login - partial; token incompatible with middleware fallback; logs user data.
- GET /api/courses - partial; depends on missing courses table schema.
- GET /api/courses/:id - partial.
- POST /api/courses/add - unprotected course creation.
- GET /api/lectures - dummy/empty response.

- POST /api/lectures/upload - no persistence, storage, auth, or upload security.

- GET /api/admin/test - test only.

- GET /api/admin/dashboard - route conflict/duplication under /api/admin.

- GET /api/admin/teachers - unprotected.

- POST /api/admin/add-teacher - unprotected.

- POST /api/admin/add-course - unprotected.

- PUT /api/admin/update-password - unprotected; email supplied by client.

- DELETE /api/admin/delete-course/:id - unprotected.

- PUT /api/admin/edit-course/:id - unprotected.

- GET /api/students - unprotected.

- POST /api/students - unprotected; stores plaintext password.

- GET /api/teachers - unprotected.

- POST /api/payment/create-order - no auth, verification, persistence, or webhook.

- GET /api/payment/test - registered after 404 middleware, likely unreachable.

- Defined but not mounted in runtime server: user, sections, enrollments, progress, quizzes, assignments, certificates, doubts, ai, activities.

8. Security Audit

- JWT: broken/inconsistent secrets; hardcoded secrets; no refresh/revocation.

- Password Hashing: teacher/user passwords hashed; student route and Google placeholder password are plaintext.

- Authorization/RBAC: weak; most admin APIs are unprotected.

- Input Validation: minimal; no schema validator.

- SQL Injection: parameterized queries are mostly used, which is a positive baseline.

- CORS: hardcoded origin; no environment allowlist.

- Helmet: installed but not used.

- Rate Limiting: missing.

- File Upload Security: no size/type limits, scan, storage policy, or signed URLs.

- Environment Variables/Secrets: .env files exist; hardcoded JWT and Razorpay test key are present.

- Security score: 1/10.

9. Folder Structure Audit

Good: backend has config/controllers/middleware/routes; frontend has pages/components/layouts/assets.

Weak: duplicate backend entrypoints, unused folders, no model/schema/migration layer, inconsistent route naming, duplicate landing folders, root/frontend/backend dependency confusion, and node_modules present in workspace.

Architecture readiness: 3/10.

10. Code Quality Audit

- Scalability: low - no service layer, transactions, pagination, caching, or background jobs.

- Maintainability: low-medium - readable but duplicated and inconsistent.

- Reusability: low - many static arrays and hardcoded dashboard values.

- Naming: inconsistent - mixed route naming and inconsistent field names.

- Best Practices: weak - no tests, migrations, CI, central validation, central API client, or structured logging.

- Lint result: npm run lint fails with 64 errors and 1 warning.

11. Production Readiness Score

- Frontend: 3.5/10
- Backend: 2.5/10
- Database: 1.5/10
- Security: 1/10
- Performance: 3/10
- Architecture: 3/10
- Code Quality: 2.5/10
- DevOps: 1.5/10
- Overall: 2.5/10

12. Missing Features

Major missing production features include schema/migrations, complete auth lifecycle, RBAC everywhere, student/teacher/admin role workflows, real curriculum model, lecture persistence/streaming, verified payment enrollment, Razorpay signature/webhook handling, progress write APIs, real dashboards, real reports, certificate eligibility, notifications, attendance, reviews, notes/download management, audit logs, tests, CI/CD, monitoring, backups, rate limiting, production logging, file scanning, pagination, search, and admin approval workflows.

13. Critical Priority Tasks

- 1. Choose one backend entrypoint and merge/remove the other.
- 2. Fix JWT signing/verifying to use one JWT_SECRET.
- 3. Protect every admin mutation route.
- 4. Add RBAC to student/teacher/admin APIs.
- 5. Hash student passwords or remove separate student auth table.
- 6. Create database migrations.
- 7. Define real schemas with foreign keys.
- 8. Normalize users/students/teachers.
- 9. Add unique constraints for email/student IDs.
- 10. Add validation middleware.
- 11. Add global error handler with safe responses.
- 12. Remove hardcoded JWT secrets.
- 13. Remove hardcoded Razorpay key from checkout.
- 14. Implement Razorpay order persistence.
- 15. Verify Razorpay payment signatures.
- 16. Add Razorpay webhooks.
- 17. Activate enrollment only after verified payment.
- 18. Implement enrollments route in runtime server.
- 19. Fix enrollment undefined addActivity/course_id bug.
- 20. Mount or delete unused route modules.
- 21. Implement lecture persistence.
- 22. Add Cloudinary upload integration or signed storage.
- 23. Add file size/type limits.
- 24. Add upload authorization.
- 25. Implement section/chapter retrieval.
- 26. Implement progress update endpoint.
- 27. Implement continue-learning resume logic.

- 28. Build real quiz frontend and APIs.
- 29. Build real assignment frontend and APIs.
- 30. Implement certificate eligibility rules.
- 31. Fix malformed frontend URLs.
- 32. Use one API base client.
- 33. Protect all frontend admin/teacher/student routes.
- 34. Fix ESLint errors.
- 35. Replace or clearly label static dashboard metrics.
- 36. Add pagination/search on list APIs.
- 37. Add Helmet.
- 38. Add rate limiting.
- 39. Configure CORS via environment allowlist.
- 40. Add request logging.
- 41. Add audit logs for admin actions.
- 42. Add tests for auth and payments.
- 43. Add tests for course/enrollment flows.
- 44. Add CI build/lint/test pipeline.
- 45. Add .env.example.
- 46. Ensure .env is ignored at every project level.
- 47. Remove generated dependency folders from source control if tracked.
- 48. Add separate backend deployment config.
- 49. Add monitoring/error tracking.
- 50. Add PostgreSQL backup/restore strategy.

14. Final Verdict

Project level: Beginner to early Intermediate prototype.

Production ready: No.

Approximate overall completion: 28%. Approximate remaining work: 72%.

Estimated time to production-ready: 8-14 weeks for a small focused team, longer for a solo developer.